GURU GHASIDAS UNIVERSITY

OBJECT ORIENTED PROGRAMMING WITH JAVA

ASSIGNMENT

Year : 2025 - 26

NAME: Aditi Pandey (4th sem)
COURSE: Bachelor of Science in Computer Science
ROLL NUMBER: 24070104
ENROLMENT NUMBER: GGV/24/05104
SUBMITTED TO: Dr. Ratnesh Srivastava

INDEX

No.	PROGRAM TITLE
1	Farmer Management System
2	Fruit Quantity & Quality System
3	Orchard Tree Representation
4	Graph Subtrees
5	Path Finding in Graph
6	Shortest Path
7	Exception Handling
8	Bank System
9	Train Platform Synchronization
10	Restaurant System

Question 01

Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

CODE :

```
import java.util.*;
import java.io.*;
// Class for Farmer
class Farmer {
    String name, fruit, city;
    int quantity;
    Farmer(String n, String f, String c, int q) {
        name = n;
        fruit = f;
        city = c;
        quantity = q;
    }
}
public class FarmerSystem {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        ArrayList<Farmer> list = new ArrayList<>();
        // INPUT 10 farmers
        for (int i = 0; i < 10; i++) {
            System.out.println("\nEnter details of Farmer " + (i + 1));
            System.out.print("Name: ");
            String name = sc.next();
            System.out.print("Fruit: ");
            String fruit = sc.next();
            System.out.print("City: ");
            String city = sc.next();
            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            list.add(new Farmer(name, fruit, city, quantity));
        }
        // SAVE BEFORE SORTING
        saveToFile("before.txt", list);
        // SORT (Simple Bubble Sort - easy to understand)
        for (int i = 0; i < list.size(); i++) {
            for (int j = 0; j < list.size() - 1; j++) {
                if (list.get(j).quantity > list.get(j + 1).quantity) {
                    Farmer temp = list.get(j);
                    list.set(j, list.get(j + 1));
                    list.set(j + 1, temp); }}}
```

```
        // SAVE AFTER SORTING
        saveToFile("after.txt", list);
        // DISPLAY SORTED DATA
        System.out.println("\nSorted Data:");
        System.out.println("Name\tFruit\tCity\tQuantity");
        for (Farmer f : list) {
            System.out.println(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity);
        }
        // SEARCH
        System.out.print("\nEnter farmer name to search: ");
        String search = sc.next();
        boolean found = false;
        for (Farmer f : list) {
            if (f.name.equalsIgnoreCase(search)) {
                System.out.println("\nFarmer Found:");
                System.out.println("Name\tFruit\tCity\tQuantity");
                System.out.println(f.name + "\t" + f.fruit + "\t" + f.city + "\t" +
f.quantity);
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Farmer not found!");

        }
        sc.close();
    }
    // FUNCTION TO SAVE DATA INTO FILE
    static void saveToFile(String fileName, ArrayList<Farmer> list) throws Exception {
        FileWriter fw = new FileWriter(fileName);
        fw.write("Name\tFruit\tCity\tQuantity\n");
        for (Farmer f : list) {
            fw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity + "\n");
        }
        fw.close();
    }
}
```

INPUT :

```
Enter details of Farmer 1
Name: Ramesh
Fruit: Mango
City: Bhopal
Quantity: 500

... (total 10 farmers)
```

OUTPUT :

```
Sorted Data:
Name   Fruit   City   Quantity
Suresh  Apple   Indore  300
Ramesh  Mango   Bhopal  500
...

Enter farmer name to search: Ramesh

Farmer Found:
Name   Fruit   City   Quantity
Ramesh  Mango   Bhopal  500
```

Question 02

Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

CODE :

```
import java.util.*;
class Fruit {
    String name;
    int quantity;
    String quality;
    Fruit(String name, int quantity, String quality) {
        this.name = name;
        // if quantity > 1000, store half
        if (quantity > 1000)
            this.quantity = quantity / 2;
        else
            this.quantity = quantity;
        this.quality = quality;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] names = {"Mango", "Apple", "Papaya"};
        Fruit[] f = new Fruit[3];
        // INPUT
```

```
        for (int i = 0; i < 3; i++) {
            System.out.println("Enter details for " + names[i]);
            System.out.print("Quantity: ");
            int qty = sc.nextInt();
            System.out.print("Quality (good/bad/average): ");
            String quality = sc.next();
            f[i] = new Fruit(names[i], qty, quality);
        }
        // DISPLAY DATA
        System.out.println("\nFruit Data:");
        for (int i = 0; i < 3; i++) {
            System.out.println(f[i].name + " " + f[i].quantity + " " + f[i].quality);
        }
        // SEARCH
        System.out.print("\nEnter fruit name to search: ");
        String searchName = sc.next();
        System.out.print("Enter quality: ");
        String searchQuality = sc.next();
        boolean found = false;
        for (int i = 0; i < 3; i++) {
            if (f[i].name.equalsIgnoreCase(searchName) &&
                f[i].quality.equalsIgnoreCase(searchQuality)) {
                System.out.println("Found: " + f[i].name + " " + f[i].quantity + " " + f[i].quality);
                found = true;
                break;
            }
        }
        if (!found)
            System.out.println("No matching fruit found!");
        sc.close();
    }
```

INPUT :

```
Enter details for Mango
Quantity: 1200
Quality: good

Enter details for Apple
Quantity: 800
Quality: average

Enter details for Papaya
Quantity: 1500
Quality: bad

Enter fruit name to search: Mango
Enter quality: good
```

OUTPUT :

```
Fruit Data:
Mango 600 good
Apple 800 average
Papaya 750 bad

Found: Mango 600 good
```

Question 03

There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

CODE :

```
import java.util.*;
class Orchard {
    String name, fruit, quality;
    Orchard(String n, String f, String q) {
        name = n;
        fruit = f;
        quality = q;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Creating orchards
        Orchard[] o = new Orchard[4];
        o[0] = new Orchard("A", "Apple", "Good");
        o[1] = new Orchard("B", "Litchi", "Good");
        o[2] = new Orchard("C", "Mango", "Good");
        o[3] = new Orchard("D", "Apple", "Bad");
        // Display tree structure
        System.out.println("Orchard Tree:");
        System.out.println("A -> Apple (Good)");
        System.out.println("B -> Litchi (Good)");
        System.out.println("C -> Mango (Good)");
        System.out.println("D -> Apple (Bad)");
        // SEARCH
```

```
        System.out.print("\nEnter fruit to search: ");
        String search = sc.next();
        boolean found = false;
        System.out.println("\nResult:");
        for (int i = 0; i < 4; i++) {
            if (o[i].fruit.equalsIgnoreCase(search)) {
                System.out.println("Orchard: " + o[i].name);
                System.out.println("Fruit: " + o[i].fruit);
                System.out.println("Quality: " + o[i].quality);
                System.out.println("----------------");
                found = true;
            }
        }
        if (!found)
            System.out.println("Fruit not found!");
        sc.close();
    }
}
```

INPUT :

```
Enter fruit to search: Apple
```

OUTPUT :

```
Orchard Tree:
A -> Apple (Good)
B -> Litchi (Good)
C -> Mango (Good)
D -> Apple (Bad)

Result:
Orchard: A
Fruit: Apple
Quality: Good
----------------
Orchard: D
Fruit: Apple
Quality: Bad
----------------
```

Question 04

Take a map provided as a graph & display all the subtrees of connected points.

CODE :

```
import java.util.*;
// Graph class
class Graph {
    int V;
    ArrayList<ArrayList<Integer>> adj;
    // Constructor
    Graph(int V) {
        this.V = V;
        adj = new ArrayList<>();
        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }
    }
    // Add edge (undirected graph)
    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }
    // DFS function
    void dfs(int node, boolean visited[]) {
        visited[node] = true;
        System.out.print(node + " ");
        for (int i : adj.get(node)) {
            if (!visited[i]) {
                dfs(i, visited);
            }
        }
    }
    // Function to print all subtrees (connected components)
    void printSubtrees() {
        boolean visited[] = new boolean[V];
        System.out.println("Connected Subtrees in Graph:");
        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                dfs(i, visited);
                System.out.println(); // new subtree
            }
        }
    }
}
public class GraphSubtrees {
    public static void main(String[] args) {
```

```
        // Creating graph with 6 nodes (0 to 5)
        Graph g = new Graph(6);
        // Adding edges (map connections)
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 2);
        g.addEdge(3, 4);
        // Node 5 is isolated
        // Display subtrees
        g.printSubtrees();
    }
}
```

INPUT :

No input is required for this program as all values are predefined.

OUTPUT :

```
Connected Subtrees in Graph:
0 1 2
3 4
5
```

Question 05

Enter entry point and goal point and produce the matching subtree of the graph.

CODE :

```
import java.util.*;
// Graph class
class Graph {
    int V;
    ArrayList<ArrayList<Integer>> adj;
    Graph(int V) {
        this.V = V;
        adj = new ArrayList<>();
        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }
    }
    // Add edge
    void addEdge(int u, int v) {
```

```
        adj.get(u).add(v);
        adj.get(v).add(u);}
    // DFS to find path
    boolean dfs(int node, int goal, boolean visited[],
ArrayList<Integer> path) {
        visited[node] = true;
        path.add(node);
        // If goal found
        if (node == goal) {
            return true; }
        for (int i : adj.get(node)) {
            if (!visited[i]) {
                if (dfs(i, goal, visited, path)) {
                    return true;
                }
            }
        }
        // Backtrack if not found
        path.remove(path.size() - 1);
        return false; }
    // Find path function
    void findPath(int start, int goal) {
        boolean visited[] = new boolean[V];
        ArrayList<Integer> path = new ArrayList<>();
        if (dfs(start, goal, visited, path)) {
            System.out.println("Path from Entry to Goal:");
            for (int node : path) {
                System.out.print(node + " ");        }
        } else {
            System.out.println("No path found between nodes.");} } }
public class GraphPath {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Graph g = new Graph(6);
        // Creating graph connections
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);
        g.addEdge(4, 5);
        System.out.print("Enter Entry Node: ");
        int start = sc.nextInt();
        System.out.print("Enter Goal Node: ");
        int goal = sc.nextInt();
        g.findPath(start, goal);
        sc.close(); }}
```

INPUT :

```
Enter Entry Node: 0
Enter Goal Node: 5
```

OUTPUT :

```
Enter Entry Node: 0
Enter Goal Node: 5
```

Question 06

Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

CODE :

```
import java.util.*;
public class SimpleShortestPath {
    static final int INF = 99999;
    // Function to find node with minimum distance
    static int minDistance(int dist[], boolean visited[], int V) {
        int min = INF, index = -1;
        for (int i = 0; i < V; i++) {
            if (!visited[i] && dist[i] <= min) {
                min = dist[i];
                index = i;
            }
        }
        return index;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int V = 5;
// number of nodes
        int graph[][] = {
            {0, 2, 4, INF, INF},
            {2, 0, 1, 7, INF},
            {4, 1, 0, INF, 3},
            {INF, 7, INF, 0, 1},
            {INF, INF, 3, 1, 0}
        };
        int dist[] = new int[V];
        boolean visited[] = new boolean[V];
        Arrays.fill(dist, INF);
        System.out.print("Enter Start Node: ");
        int start = sc.nextInt();
```

```
        System.out.print("Enter End Node: ");
        int end = sc.nextInt();
        dist[start] = 0;
        // Simple Dijkstra logic
        for (int i = 0; i < V - 1; i++) {
            int u = minDistance(dist, visited, V);
            visited[u] = true;
            for (int v = 0; v < V; v++) {
                if (!visited[v] &&
                    graph[u][v] != 0 &&
                    graph[u][v] != INF &&
                    dist[u] != INF &&
                    dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
        System.out.println("\nShortest Distance from " + start + " to " +
end + " is: " + dist[end]);
        sc.close();
    }
}
```

INPUT :

```
Enter Start Node: 0
Enter End Node: 4
```

OUTPUT :

```
Shortest Distance from 0 to 4 is: 6
```

Question 07

Write a program to demonstrate the use of all keywords made for exception handling.

CODE :

```
import java.util.*;
public class ExceptionDemo {
    // Method using throws
    static void divide(int a, int b) throws ArithmeticException {
        // Using throw keyword
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");}
```

```
        System.out.println("Result: " + (a / b));    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();
            // Method call that may throw exception
            divide(a, b);        }
        // catch block
        catch (ArithmeticException e) {
            System.out.println("Exception caught:"+ e.getMessage());     }
        // finally block
        finally {
            System.out.println("Program execution completed (finally block always runs).");
}
        sc.close();
}
}
```

INPUT :

```
Enter first number: 10
Enter second number: 0
```

OUTPUT :

```
Exception caught: Cannot divide by zero
Program execution completed (finally block always runs).
```

Question 08

Write a program to withdraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

CODE :

```
import java.util.*;
// Custom Exception class
class NoMoneyException extends Exception {
    NoMoneyException(String message) {
        super(message);
    }
}
// Bank Account class
```

```
class BankAccount {
    int balance = 5000;
    // Withdraw method
    void withdraw(int amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException("Insufficient Balance! Withdrawal not
allowed.");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawal successful!");
            System.out.println("Remaining Balance: " + balance);
        }
    }
}
public class BankSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount acc = new BankAccount();
        try {
            System.out.print("Enter amount to withdraw: ");
            int amount = sc.nextInt();
            acc.withdraw(amount);
        }
        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            System.out.println("Transaction completed.");
        }
        sc.close();
    }
}
```

INPUT :

```
Enter amount to withdraw: 7000
```

OUTPUT :

```
Exception: Insufficient Balance! Withdrawal not allowed.
Transaction completed.
```

Question 09

Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

CODE :

```
class Platform {
    // synchronized method so only one train can enter
    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the platform.");
        try {
            // simulate time taken on platform
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println(trainName + " is leaving the platform.");
    }
}
// Train class (Thread)
class Train extends Thread {
    Platform p;
    String name;
    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }
    public void run() {
        p.usePlatform(name);
    }
}
public class TrainSystem {
    public static void main(String[] args) {
        Platform platform = new Platform();
        // Multiple trains
        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");
        // Start threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

INPUT :

No input is required for this program as all values are predefined.

OUTPUT :

```
Train A is entering the platform.
Train A is leaving the platform.
Train B is entering the platform.
Train B is leaving the platform.
Train C is entering the platform.
Train C is leaving the platform.
```

Question 10

Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

CODE :

```
class Kitchen {
    // Only one order can be processed at a time
    synchronized void prepareOrder(String order, String waiterName) {
        System.out.println(waiterName + " placed order: " + order);
        try {
            // simulate cooking time
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println("Chef prepared: " + order + " for " + waiterName);
        System.out.println("-----------------------------------");
    }
}
// Waiter class (Thread)
class Waiter extends Thread {
    Kitchen kitchen;
    String waiterName;
    String order;
    Waiter(Kitchen kitchen, String waiterName, String order) {
        this.kitchen = kitchen;
        this.waiterName = waiterName;
        this.order = order;
    }
    public void run() {
        kitchen.prepareOrder(order, waiterName);
```

```
    }
}
public class RestaurantSystem {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();
        // Multiple waiters placing orders

        Waiter w1 = new Waiter(kitchen, "Waiter A", "Pizza");

        Waiter w2 = new Waiter(kitchen, "Waiter B", "Burger");
        Waiter w3 = new Waiter(kitchen, "Waiter C", "Pasta");
        // Start threads

        w1.start();
        w2.start();
        w3.start();
}
}
```

INPUT :

No input is required for this program as all values are predefined.

OUTPUT :

```
Waiter A placed order: Pizza
Chef prepared: Pizza for Waiter A
----------------------------------
Waiter B placed order: Burger
Chef prepared: Burger for Waiter B
----------------------------------
Waiter C placed order: Pasta
Chef prepared: Pasta for Waiter C
----------------------------------
```